ASPN Platform Development Manual

Initial Prototype, Codex Revision Phase, and September MVP Backend Foundation

Working Version: June 2026

---

1. Purpose of This Manual

This manual documents the early development of the Aspirations Network (ASPN) Platform, beginning with the initial prototype codebase and continuing through the structured Codex-assisted backend revision process.

The purpose of this document is to create a clear record of:

1. What the original ASPN platform prototype contained.
2. Why the platform was revised.
3. How Codex was used to revise the backend in controlled checkpoints.
4. What changes were made at each checkpoint.
5. What the platform can now support.
6. What still needs to be built before youth onboarding.
7. What testing steps were used.
8. What future developers, staff, or project partners need to understand.

This document is intended to serve as a detailed internal development manual. Shorter versions may later be created for board members, funders, implementation partners, dissertation documentation, or future developers.

PART I: ASPN PLATFORM CONTEXT

---

2. Organizational Context

Aspirations Network (ASPN) is a youth development and civic engagement organization based in the St. Louis region.

ASPN programming is designed to support K–12 students, college students, and emerging adults, generally ages 8–24. The organization's broader work includes youth leadership development, civic engagement, community service, civic media, research participation, and workforce readiness.

Current and planned ASPN programming includes:

- Youth2Lead
- Youth Advisory Boards
- Civic engagement programming
- Community service projects
- Civic media production
- Research participation opportunities
- Revolution Will Be Digitized (RWD)
- Civic Fellows
- Future Ready workforce pathway initiatives

The ASPN Platform is intended to serve as the digital infrastructure supporting these programs.

---

3. Platform Vision

The ASPN Platform is a youth development, credentialing, civic engagement, workforce readiness, and opportunity platform.

The platform is designed to help young people:

- Create private profiles
- Document accomplishments
- Earn and display credentials
- Participate in ASPN programs
- Track attendance
- Track service-hour verification
- Build civic knowledge
- Develop leadership skills
- Connect participation to future educational and workforce opportunities

The platform is not intended to function as a general social media platform.

It should not be designed to maximize:

- Screen time
- Viral engagement
- Algorithmic recommendation loops
- Public popularity
- Follower growth
- Advertising revenue

The platform should instead feel more like:

- A developmental portfolio
- A credential wallet
- A program participation system
- A civic opportunity network
- A youth-safe learning and participation environment

---

4. Core Design Principles

The platform should prioritize:

1. Youth safety
2. Privacy
3. Simplicity
4. Accessibility
5. Educational value
6. Staff oversight
7. Long-term scalability
8. Maintainable architecture

When safety conflicts with engagement, the platform should choose safety.

When privacy conflicts with convenience, the platform should choose privacy.

When automation conflicts with human oversight, the platform should choose human oversight.

---

5. Features Intentionally Out of Scope for the September MVP

Several features may be possible in the future, but they were intentionally excluded from the September MVP backend foundation.

Do not build for the September MVP:

- Direct messaging
- Peer-to-peer messaging
- Public youth profiles
- Public youth profile discovery
- Public youth search
- Follower systems
- Friend systems
- Popularity rankings
- Like-driven engagement systems
- Infinite scrolling
- Viral recommendation systems
- Public discussion features without moderation structure

These features are not necessarily permanently prohibited. However, they should only be revisited after stronger safety, moderation, consent, privacy, and role-based access systems exist.

PART II: INITIAL PROTOTYPE

---

6. Initial Prototype Overview

The initial ASPN platform prototype was a Java Spring Boot backend connected to Firebase and Firestore.

The original backend supported a basic community platform structure with user profiles and discussion-board functionality.

The initial codebase included:

- Spring Boot backend
- Firebase Admin SDK
- Firestore database access
- User profile creation
- User profile retrieval
- Discussion post creation
- Discussion post retrieval
- Comment creation

- Comment retrieval
- Post upvoting
- Post deletion

The initial platform resembled a basic authenticated community or discussion platform rather than a full youth credentialing, program participation, or workforce pipeline platform.

---

7. Original Technical Architecture

The original backend architecture used:

- Java 21
- Spring Boot
- Gradle
- Firebase Admin SDK
- Google Firestore
- Firebase Hosting connection from the broader app context

The backend project was located locally at:

```
None
/Users/colbycrowder/Documents/AspirationsBackendColby/UserData
```

The GitHub repository was:

```
None
colbycrowder/AspirationsBackendColby
```

The main project folder contained:

```
None
build.gradle
Dockerfile
gradlew
gradlew.bat
settings.gradle
src
```

---

8. Original Main Backend Components

The original backend included the following major files and components:

Application

- `UserDataApplication.java`

This was the main Spring Boot application entry point.

Configuration

- `FireBaseConfig.java`

This configured Firebase and Firestore access using Google application-default credentials.

Controller

- `UserInfoController.java`

This exposed `/api/...` routes for user profiles, discussion posts, comments, upvotes, and deletion.

Services

- `UserInfoService.java`
- `DiscussionPostService.java`

These handled core backend business logic and Firestore operations.

Models

- `User.java`
- `DiscussionPost.java`
- `Comment.java`

These represented user profile data, discussion posts, and post comments.

DTOs

- `UserProfileCreationDTO.java`

This represented user profile creation input.

Tests

- `UserDataApplicationTests.java`

The initial test suite only included a basic Spring context-loading test.

---

9. Original Firestore Collections

The initial backend used or referenced the following Firestore collections:

Collection	Original Purpose
`aspirationnetworkusers`	Primary user profile collection
`aspirationnetworkposts`	Discussion post collection
`comments`	Comment collection
`users`	Incorrectly referenced during post creation

One early issue was that user profiles were created in `aspirationnetworkusers`, but discussion post creation attempted to read user data from `users`. This created a collection mismatch and could prevent creator names from appearing correctly.

---

10. Original API Endpoints

The initial backend exposed endpoints including:

Method	Path	Purpose
`GET`	`/api/getUser/{id}`	Fetch user profile by UID
`POST`	`/api/createProfile`	Create user profile
`GET`	`/api/getallpost`	Fetch all discussion posts

`POST`	`/api/creatPost`	Create discussion post; typo in route
`DELETE`	`/api/deletePost/{postID}`	Delete a post
`POST`	`/api/upVote/{postID}`	Increment post upvotes
`POST`	`/api/createComment`	Create a comment
`GET`	`/api/getCommentForPost/{postID}`	Fetch comments for a post

---

11. Initial Prototype Strengths

The initial prototype had several important strengths:

1. It already used a real backend framework.
2. It was connected to Firebase/Firestore.
3. It had a working user profile concept.
4. It had a basic content/community model.
5. It was small enough to revise quickly.
6. It could be preserved rather than discarded.
7. It provided a foundation for a more structured youth platform.

This meant the project did not need to restart from zero.

---

12. Initial Prototype Limitations

The initial prototype was not ready for September youth onboarding.

Key limitations included:

Authentication and Authorization

The backend did not yet visibly enforce Firebase ID token verification or staff/youth role boundaries across sensitive routes.

Credentialing

There was no formal credential system. The original `User` model included a `badges` field, but this was not enough for a serious credentialing platform.

Attendance

There was no attendance record model or attendance workflow.

Service Hours

There was no service-hour verification record model or workflow.

Staff Management

There was no staff-protected user management system.

Profile Privacy

The original platform needed clearer private-by-default profile logic.

Error Handling

Some controller error responses attempted unsafe exception casting.

Deletion Logic

Post deletion did not properly delete all related comments.

Testing

Only a basic context-loading test existed, and it initially failed locally because Firebase credentials were not configured.

PART III: DEVELOPMENT ENVIRONMENT SETUP

---

13. GitHub and Local Repository Setup

The project was connected to GitHub through the repository:

```
None
colbycrowder/AspirationsBackendColby
```

The repository was cloned locally into:

```
None
/Users/colbycrowder/Documents/AspirationsBackendColby
```

The backend project was inside:

```
None
/Users/colbycrowder/Documents/AspirationsBackendColby/UserData
```

Git was verified using:

```
Shell
git remote -v
```

The repository remote was confirmed as:

```
None
origin https://github.com/colbycrowder/AspirationsBackendColby.git
```

---

14. Codex Environment Setup

Codex was connected to the local repository environment.

The correct Codex environment path was:

```
None
/Users/colbycrowder/Documents/AspirationsBackendColby
```

Codex was able to see:

- `UserData`

- `build.gradle`
- `gradlew`
- `Dockerfile`
- `settings.gradle`
- `src`

This confirmed Codex was reading the correct repository rather than the separate ASPN planning folder.

---

15. Java and Gradle Setup

The backend required Java 21.

Initially, Terminal could not locate Java. Java 21 was then installed using Eclipse Temurin.

Java installation was verified with:

```shell
java -version
```

The successful Java version output showed:

```none
openjdk version "21.0.11"
```

The project was then tested using:

```shell
cd "/Users/colbycrowder/Documents/AspirationsBackendColby/UserData"
./gradlew clean test
```

Gradle downloaded version `8.14.3` and successfully compiled the code.

---

16. Initial Local Test Result

The first local test did not pass because Firebase/Google application-default credentials were not configured locally.

The key error was:

```
None
Caused by: java.io.IOException at
DefaultCredentialsProvider.java:127
```

This meant:

- Java worked.
- Gradle worked.
- The code compiled.
- The failure came from Firebase credentials, not from broken Java code.

This led to the first testing setup revision.

PART IV: CODEX REVISION PROCESS

---

17. Revision Philosophy

The backend was revised through a checkpoint-based process instead of allowing Codex to rewrite the entire codebase at once.

This workflow was chosen because the user is not a programmer and needed to verify each major development step before moving forward.

The process used the following pattern:

1. Codex completes one checkpoint.
2. Codex reports files changed.
3. Codex explains the changes in plain English.
4. Codex provides the local test command.
5. The user verifies `BUILD SUCCESSFUL`.
6. Only then does the user move to the next checkpoint.

This avoided large uncontrolled revisions and made the development process more understandable, safer, and easier to document.

---

18. Standard Local Test Command

After each checkpoint, the same local test command was used:

```shell
cd "/Users/colbycrowder/Documents/AspirationsBackendColby/UserData"
./gradlew clean test
```

A successful checkpoint required:

```none
BUILD SUCCESSFUL
```

---

19. Pre-Checkpoint: Testing Setup

Purpose

The first change was not a feature change. It was a testing setup correction.

The goal was to allow basic local tests to pass without requiring live Firebase or Google credentials.

Why This Was Needed

The default Spring Boot context test tried to start the full app context, including Firebase/GCP configuration. Since the local machine did not have Google application-default credentials configured, the test failed.

Files Changed

- `UserDataApplicationTests.java`

What Changed

Codex changed the test setup to:

- Load a narrower Spring Boot test context.
- Use mocked `FirebaseApp` and `Firestore` beans.
- Disable Spring Cloud GCP Firestore and Storage auto-configuration only for the test context.

Test properties included:

```
None
spring.cloud.gcp.firestore.enabled=false
spring.cloud.gcp.storage.enabled=false
```

Why It Matters

This allowed:

```
Shell
./gradlew clean test
```

to pass locally without connecting to live Firestore.

What Was Not Changed

- Production Firebase behavior was not changed.
- API behavior was not changed.
- No credentials were added.
- No live Firestore connection was introduced.

Result

Local tests passed with:

```
None
BUILD SUCCESSFUL
```

PART V: CHECKPOINT DOCUMENTATION

---

20. Checkpoint 1: Backend Cleanup and Safety Fixes

Purpose

Checkpoint 1 cleaned up issues in the existing backend without adding new features.

The goal was to stabilize the foundation before adding credentials, attendance, or service-hour records.

Files Changed

- `UserInfoController.java`
- `DiscussionPostService.java`
- `UserInfoService.java`
- `FireBaseConfig.java`
- `UserInfoControllerTest.java`
- `UserDataApplicationTests.java`

What Changed

Codex made the following revisions:

1. Fixed unsafe controller error responses.
2. Added a correctly spelled endpoint:

```
None

/api/createPost
```

3. Preserved the old typo endpoint:

```
None

/api/creatPost
```

4. Fixed the Firestore user collection mismatch.

5. Updated post creation to use:

```
None

aspirationnetworkusers
```

6. Fixed post deletion so related comments are handled correctly.
7. Removed unused imports and fields.
8. Added controller tests for create-post behavior and safe error responses.

Why It Matters

This checkpoint corrected backend issues that could cause runtime errors or data inconsistency.

For September onboarding, this made the backend more stable before adding profile and credential features.

What Was Not Built

- Credentials
- Attendance
- Service hours
- Profile expansion
- Staff dashboard
- Public profiles
- Peer-to-peer features

Risks and Cautions

The old typo endpoint remained intentionally for compatibility.

Firestore batch deletes may eventually need pagination if a post has hundreds of comments.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

21. Checkpoint 2: Profile Foundation

Purpose

Checkpoint 2 strengthened the user profile model for the September MVP.

The goal was to make youth profiles the central record for credentials, program participation, attendance, and service-hour tracking.

Files Changed

- `User.java`
- `UserInfoService.java`
- `UserInfoServiceTest.java`

What Changed

Codex added private-by-default profile fields and foundation fields for future program participation.

New or strengthened profile concepts included:

- Program participation
- Credentials
- Attendance records
- Service-hour records
- Staff review fields
- Staff verification fields
- Private profile defaults
- `profileStatus = "pending_onboarding"`

New users were created as private youth profiles.

Why It Matters

This checkpoint shifted the platform from a basic profile/discussion prototype toward a youth program management platform.

It gave ASPN profiles the structure needed to support:

- September onboarding
- Credential display
- Attendance tracking
- Service-hour tracking

- Staff verification

What Youth Users Could Do After This Checkpoint

Youth users did not receive a new visible workflow yet.

However, newly created youth profiles began storing safer MVP-ready defaults.

What ASPN Staff Could Do After This Checkpoint

No new staff workflow was added yet.

The backend profile structure was prepared for future staff review and verification.

What Was Not Built

- Public profiles
- Peer-to-peer features
- Credential awarding workflow
- Attendance workflow
- Service-hour workflow
- Staff dashboard
- Firebase security rule updates

Risks and Cautions

Existing Firestore users may not automatically have the new fields.

Older documents may return `null` for newer fields unless defaults or migrations are handled later.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

22. Checkpoint 3: Credential Foundation

Purpose

Checkpoint 3 created the core backend credential data structure.

The goal was to build the credential container without requiring ASPN to finalize the names, icons, or catalog of credentials yet.

Files Changed

- `CredentialDefinition.java`
- `EarnedCredential.java`
- `User.java`
- `UserInfoService.java`
- `CredentialModelTest.java`
- `UserInfoServiceTest.java`

What Changed

Codex added a generic credential foundation with two key models:

CredentialDefinition

This represents the type of credential that can exist.

Supported fields included:

- `credentialID`
- `credentialName`
- `description`
- `icon`
- `category`
- `active`
- timestamps

EarnedCredential

This connects a youth user to a credential definition.

Supported fields included:

- user UID
- credential ID
- staff-award metadata
- earned/issued data

User profiles were updated to include:

```
None
earnedCredentialIds
```

The existing `badges` field was preserved for compatibility.

Why It Matters

This checkpoint allowed ASPN to define credentials later without hardcoding names into the backend.

This means ASPN can later add credentials such as:

- Attendance credentials
- Youth2Lead credentials
- RWD video credentials
- Committee credentials
- Service credentials
- Civic project credentials
- Workforce pathway credentials

without redesigning the backend.

What Youth Users Could Do After This Checkpoint

Nothing new directly.

Youth users still could not self-award credentials.

What ASPN Staff Could Do After This Checkpoint

No staff workflow was built yet.

The backend models now supported future staff-created credential definitions and earned credential records.

What Was Not Built

- Credential catalog
- Staff credential dashboard
- Credential awarding endpoint
- Attendance
- Service-hour verification
- Public profiles
- Peer-to-peer features

Risks and Cautions

Existing users may have:

```
None
earnedCredentialIds = null
```

until migrated, updated, or recreated.

Credential validation and permission checks still needed to be built before real awarding.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

23. Checkpoint 4: Staff Credential Management

Purpose

Checkpoint 4 made credentials usable by staff at the backend level.

The goal was to create staff-facing backend routes for credential definition creation and credential awarding.

Files Changed

- `UserInfoController.java`
- `CredentialService.java`
- `CredentialDefinitionCreationDTO.java`
- `AwardCredentialDTO.java`
- `CredentialDefinition.java`
- `UserDataApplicationTests.java`
- `UserInfoControllerTest.java`
- `CredentialServiceTest.java`

What Changed

Codex added staff-prefixed backend foundations:

```
None
POST /api/staff/credentials/definitions
POST /api/staff/credentials/award
```

Credential definitions are created from staff-provided values only.

No credential names, descriptions, icons, or catalog records were hardcoded.

Credential awarding now:

1. Creates an `earnedCredentials` record.
2. Associates it with a youth user profile through `earnedCredentialIds`.

Why It Matters

This was the first checkpoint where the backend could structurally support the real credential workflow:

```
None
Staff creates credential definition
↓
Staff awards credential
↓
Youth profile is associated with earned credential
```

This is central to the September onboarding goal.

What Youth Users Could Do After This Checkpoint

Nothing new directly.

Youth users still could not self-award credentials.

What ASPN Staff Could Do After This Checkpoint

Structurally, staff-facing API methods existed to:

- Create credential definitions
- Award credentials to youth users

What Was Not Built

- Firebase token verification
- Staff role enforcement
- Full credential dashboard
- Credential catalog seed data
- Attendance
- Service-hour verification
- Public profiles
- Peer-to-peer features

Firestore Collections Added or Expected

- `credentialDefinitions`
- `earnedCredentials`

Awarding credentials also updates youth documents in:

```
None
aspirationnetworkusers
```

Risks and Cautions

At this stage, the routes were structured as staff-only but were not yet protected by Firebase token and role checks.

They should not be exposed in production until authentication and authorization are added.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

24. Checkpoint 5: Credential Display and Profile Retrieval

Purpose

Checkpoint 5 allowed profiles to retrieve earned credential information for display.

The goal was to prepare the backend for a profile page that shows youth credentials.

Files Changed

- `UserInfoController.java`
- `CredentialService.java`
- `EarnedCredentialDisplayDTO.java`
- `UserProfileWithCredentialsDTO.java`
- `UserInfoControllerTest.java`
- `CredentialServiceTest.java`

What Changed

Codex added a new private profile retrieval endpoint:

```
None
GET /api/getUserWithCredentials/{id}
```

This endpoint returns:

- User profile information
- Earned credential display data
- Credential name
- Credential description
- Credential icon
- Credential category
- Credential status
- Award dates

The original endpoint was preserved:

```
None
GET /api/getUser/{id}
```

Why It Matters

This created the backend foundation for displaying credentials on a youth profile.

The profile page could now show credentials without hardcoding credential content and without making profiles public.

What Youth Users Could Do After This Checkpoint

Youth profile pages could now be retrieved with earned credential display data.

Youth still could not self-award credentials.

What ASPN Staff Could Do After This Checkpoint

Staff-awarded credentials could now be reflected in a profile-ready backend response.

What Was Not Built

- Public profile discovery
- Peer-to-peer features
- Frontend dashboard
- Credential catalog seed data
- Attendance
- Service-hour verification
- Firebase token/role enforcement

Firestore Collections Used

This endpoint reads from:

- `aspirationnetworkusers`
- `earnedCredentials`
- `credentialDefinitions`

Risks and Cautions

Credential display depends on matching:

```
None
earnedCredentials.credentialID
```

to a document in:

```
None
credentialDefinitions
```

Authentication/authorization still needed to be added before production youth onboarding.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

25. Checkpoint 6: Authentication and Role Protection

Purpose

Checkpoint 6 added Firebase ID token verification and staff role protection for sensitive credential routes.

This was a major youth safety and platform security checkpoint.

Files Changed

- `FireBaseConfig.java`
- `AuthService.java`
- `UnauthorizedAccessException.java`
- `ForbiddenAccessException.java`
- `UserInfoController.java`
- `UserDataApplicationTests.java`
- `UserInfoControllerTest.java`
- `AuthServiceTest.java`

What Changed

Codex added backend Firebase ID token verification using `FirebaseAuth`.

The following staff routes were protected:

```
None
POST /api/staff/credentials/definitions
POST /api/staff/credentials/award
```

These routes now require:

```
None
Authorization: Bearer <Firebase ID token>
Content-Type: application/json
```

The backend verifies the Firebase token and then checks the user profile role in Firestore.

Allowed staff roles are:

```
None
staff
admin
```

The backend also overwrites staff UID fields using the verified Firebase token, so clients cannot spoof:

- `createdByStaffUID`
- `awardedByStaffUID`

Why It Matters

This was a key safety step.

Credential creation and awarding are sensitive actions. They should not be open to youth users or unauthenticated users.

This checkpoint ensured that only verified staff/admin users can create or award credentials.

What Youth Users Could Do After This Checkpoint

Nothing new directly.

Youth users still could not self-award credentials.

What ASPN Staff Could Do After This Checkpoint

Staff users with:

```
None
role = "staff"
```

or

```
None
role = "admin"
```

could call protected credential routes if they had a valid Firebase ID token.

What Was Not Built

- Frontend token integration
- Full staff dashboard
- Attendance
- Service-hour verification
- Peer-to-peer features
- Public profile discovery

Firebase/Staging Notes

Staff users must exist in:

```
None
aspirationnetworkusers
```

with:

```
None
role = "staff"
```

or:

```
None
role = "admin"
```

Risks and Cautions

Only the two staff credential routes were protected in this checkpoint.

Other routes still retained previous access behavior.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

26. Checkpoint 7: Staff User Management

Purpose

Checkpoint 7 added staff-protected routes for reviewing and managing youth users.

This checkpoint was important because staff need a way to view youth profiles and update onboarding-related fields.

Files Changed

- `UserInfoController.java`
- `UserInfoService.java`
- `User.java`
- `StaffUserUpdateDTO.java`
- `UserInfoControllerTest.java`
- `UserInfoServiceTest.java`

What Changed

Codex added staff-protected user management routes:

```
None
GET /api/staff/users/youth
GET /api/staff/users/youth/{id}
PATCH /api/staff/users/youth/{id}
```

The update route allows limited staff-managed fields:

- `profileStatus`
- `programIds`
- `staffReviewRequired`
- `staffVerified`

The update route does not update:

- `role`
- `publicProfile`
- Other sensitive or user-owned fields

Why It Matters

ASPN staff can now review youth profiles, see who is in onboarding, and update limited onboarding/review fields.

This supports the September onboarding process for approximately 100 youth.

What Youth Users Could Do After This Checkpoint

Nothing new directly.

Youth users cannot access staff user-management routes and cannot update their own role.

What ASPN Staff Could Do After This Checkpoint

With a valid Firebase token and staff/admin role, staff can:

- List youth users
- Retrieve one youth profile
- Update limited onboarding fields

What Was Not Built

- Attendance
- Service-hour verification
- Peer-to-peer features
- Public profile discovery
- Role-management workflow
- Staff frontend dashboard

Firebase/Staging Notes

Staff callers must have a user document in:

```
None
aspirationnetworkusers
```

with:

```
None
role = "staff"
```

or:

```
None
role = "admin"
```

Youth listing queries:

```
None
youthProfile = true
```

Older staging profiles without this field may not appear until updated or recreated.

Risks and Cautions

The update endpoint mirrors:

```
None
programIds
```

into:

```
None
programParticipationIds
```

for compatibility.

This should be normalized later.

Role updates are intentionally not included yet because they require a stricter admin-only workflow.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

27. Checkpoint 8: Attendance Foundation

Purpose

Checkpoint 8 created the backend foundation for staff-managed attendance tracking.

Files Changed

- `AttendanceRecord.java`
- `AttendanceRecordCreationDTO.java`
- `AttendanceService.java`
- `UserInfoController.java`
- `UserDataApplicationTests.java`
- `UserInfoControllerTest.java`
- `AttendanceServiceTest.java`

What Changed

Codex added an attendance foundation.

New model:

```
None
AttendanceRecord
```

New routes:

```
None
POST /api/staff/attendance
GET /api/staff/attendance/user/{userUID}
```

Attendance records store:

- User UID
- Program ID

- Event name
- Event date
- Attendance status
- Staff recorder UID
- Created timestamp
- Updated timestamp

Valid attendance statuses are:

- `present`
- `absent`
- `excused`
- `pending`

Why It Matters

ASPN now has a backend structure for staff to record program or event attendance tied to youth profiles.

Attendance is central to September program participation tracking and may later connect to credentials.

What Youth Users Could Do After This Checkpoint

Nothing new directly.

Youth self-service attendance access was not built yet.

What ASPN Staff Could Do After This Checkpoint

Staff/admin users can:

- Create attendance records
- Retrieve attendance records for a youth user

What Was Not Built

- Automated check-in
- QR-code attendance
- Youth self-view attendance route
- Attendance update workflow
- Service-hour verification
- Public profiles
- Peer-to-peer features

Firestore Collection Added or Expected

```
None
attendanceRecords
```

Creating an attendance record also updates the youth profile's:

```
None
attendanceRecordIds
```

field in:

```
None
aspirationnetworkusers
```

Risks and Cautions

Attendance retrieval is staff-protected for now to avoid broad public access.

Existing youth profiles without `attendanceRecordIds` should still work, but the field will populate as records are created.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

28. Checkpoint 9: Service-Hour Verification Foundation

Purpose

Checkpoint 9 created the backend foundation for staff-managed service-hour verification.

Files Changed

- `ServiceHourRecord.java`

- `ServiceHourRecordDTO.java`
- `ServiceHourService.java`
- `UserInfoController.java`
- `UserDataApplicationTests.java`
- `UserInfoControllerTest.java`
- `ServiceHourServiceTest.java`

What Changed

Codex added service-hour verification foundation routes:

```
None
POST /api/staff/service-hours
GET /api/staff/service-hours/user/{userUID}
```

Service-hour records support:

- Service-hour record ID
- User UID
- Program ID
- Service date
- Hours
- Description
- Verification status
- Verification source
- Reviewed by staff UID
- Submitted timestamp
- Reviewed timestamp
- Created timestamp
- Updated timestamp

Recommended verification statuses:

- `pending`
- `verified`
- `rejected`

Codex also added:

```
None
googleFormResponseUrl
```

as a safe placeholder field.

No Google Form URL was invented or hardcoded.

Why It Matters

ASPN now has the backend structure to track and review youth service hours later while keeping access staff-controlled.

This aligns with the planned workflow where youth may eventually request verification through a Google Form or other linked form process.

What Youth Users Could Do After This Checkpoint

Nothing new directly.

Youth self-submission and youth self-view were not built yet.

What ASPN Staff Could Do After This Checkpoint

Staff/admin users can:

- Create service-hour records
- Review service-hour records
- Retrieve service-hour records for a youth user

What Was Not Built

- Google Form integration
- Automated verification
- Youth submission workflow
- Youth self-view route
- Public profiles
- Peer-to-peer features

Firestore Collection Added or Expected

```
None
serviceHourRecords
```

Creating a service-hour record also updates the youth profile's:

```
None
serviceHourRecordIds
```

field in:

```
None
aspirationnetworkusers
```

Risks and Cautions

The Google Form field is only a placeholder.

Staff routes are protected by Firebase token plus staff/admin role, but staging staff users must have the correct role in Firestore.

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

29. Checkpoint 10: Youth Self-Service Profile Retrieval

Purpose

Checkpoint 10 created a protected backend route allowing a signed-in youth user to retrieve their own private profile.

This checkpoint brought together the profile, credentials, attendance, and service-hour records into one youth-facing profile retrieval endpoint.

Files Changed

- `UserInfoController.java`
- `AuthService.java`
- `YouthSelfServiceProfileDTO.java`
- `UserInfoControllerTest.java`

- `AuthServiceTest.java`

What Changed

Codex added the protected youth self-service route:

```
None
GET /api/me/profile
```

This route:

1. Verifies the Firebase ID token.
2. Uses the UID from that token.
3. Does not accept a user-provided UID.
4. Returns only that user's private profile summary.

The response includes:

- Basic private profile information
- Earned credentials
- Attendance records
- Service-hour records

Why It Matters

Youth users can now see their own onboarding-related records from one backend endpoint without exposing other youth profiles.

This is central to the September MVP goal.

What Youth Users Can Now Do

Signed-in youth users can retrieve their own private profile summary.

What ASPN Staff Can Now Do

No new staff workflow was added in this checkpoint.

What Was Not Built

- Youth profile editing
- Youth attendance submission
- Youth service-hour submission
- Google Form integration

- Public profiles/search
- Peer-to-peer features

Firebase/Staging Notes

The Firebase token UID must match a document ID in:

```
None
aspirationnetworkusers
```

Related records are read from:

- `earnedCredentials`
- `attendanceRecords`
- `serviceHourRecords`

Risks and Cautions

If a youth has no profile document yet, the endpoint returns:

```
None
404
```

If related records are missing or empty, the response should include empty lists.

Frontend Must Send

```
None
Authorization: Bearer <Firebase ID token>
Content-Type: application/json
```

Result

Codex verified:

```
None
BUILD SUCCESSFUL
```

---

30. Checkpoint 11: Staging and Firebase Readiness Review

Purpose

Checkpoint 11 was a review checkpoint, not a feature-building checkpoint.

The goal was to determine whether the backend is ready to be tested against Firebase staging/production for the September MVP.

What Was Reviewed

Codex reviewed:

1. Firebase configuration readiness
2. Firestore collections expected
3. Required staff/youth roles in Firestore
4. Required Firebase ID token behavior
5. Required frontend headers
6. Expected test users
7. Risks before Firebase deployment
8. Missing fields in older users
9. Whether emulator testing is now recommended
10. Whether staging deployment should happen before production

What Is Ready

The backend is ready for Firebase staging validation because it has:

- Firebase Admin configuration
- Firebase ID token verification
- Staff role checks
- Youth self-service profile route
- Staff-protected credential routes
- Staff-protected user management routes
- Staff-protected attendance routes
- Staff-protected service-hour routes
- Local tests passing with mocked Firebase/Firestore

What Is Not Ready

The backend is not ready for production until staging validation occurs.

Remaining concerns:

- Firestore rules have not been reviewed.
- Some public/legacy routes remain unprotected.
- Frontend token integration has not been verified.
- Older Firestore user documents may be missing new fields.
- No deployed Firebase/staging verification has been run yet.

Expected Firestore Collections

The September backend foundation expects:

- `aspirationnetworkusers`
- `aspirationnetworkposts`
- `comments`
- `credentialDefinitions`
- `earnedCredentials`
- `attendanceRecords`
- `serviceHourRecords`

Important User Fields

User documents should include:

- `uid`
- `role`
- `youthProfile`
- `publicProfile`
- `profileStatus`
- `programIds`
- `programParticipationIds`
- `earnedCredentialIds`
- `attendanceRecordIds`
- `serviceHourRecordIds`
- `staffReviewRequired`
- `staffVerified`

Required Test Accounts

At minimum, Firebase staging should include:

Youth Test User

Firestore document ID must match Firebase UID.

Required fields:

```
None
role = "member"
youthProfile = true
publicProfile = false
```

Staff Test User

Firestore document ID must match Firebase UID.

Required field:

```
None
role = "staff"
```

Admin Test User

Optional, but useful.

Required field:

```
None
role = "admin"
```

Required Frontend Headers

Protected routes must include:

```
None
Authorization: Bearer <Firebase ID token>
Content-Type: application/json
```

Suggested Staging Test Sequence

1. Deploy to Firebase staging, not production.
2. Confirm backend starts with application-default credentials.
3. Create one youth and one staff Firestore profile.
4. Test youth login.

5. Call:

```
None

GET /api/me/profile
```

6. Confirm youth cannot call staff routes.
7. Test staff routes:
    - List youth users
    - Retrieve one youth user
    - Create credential definition
    - Award credential
    - Create attendance record
    - Create service-hour record
8. Re-test `/api/me/profile` as youth.
9. Confirm credentials, attendance, and service hours appear.
10. Test missing or invalid token behavior.
11. Test staff route with `role = "member"` and confirm forbidden response.

Recommendation

Deploy to Firebase staging now.

Do not deploy to production yet.

Staging is the correct next step because the backend is structurally ready for real Firebase Auth and Firestore testing, but production safety depends on validating:

- Credentials
- Roles
- Frontend token headers
- Existing data compatibility
- Firestore rules

PART VI: CURRENT PLATFORM STATUS AFTER CHECKPOINT 11

---

31. Current Backend Capabilities

After Checkpoint 11, the backend supports the following September MVP foundation:

Youth Profile Foundation

- Youth profile creation
- Private profile defaults
- Profile onboarding status
- Program participation fields
- Credential record fields
- Attendance record fields
- Service-hour record fields

Credential Infrastructure

- Credential definition model
- Earned credential model
- Staff-created credential definitions
- Staff-awarded credentials
- Credential display retrieval
- Profile association through `earnedCredentialIds`

Staff Management

- Staff-protected youth user listing
- Staff-protected individual youth profile review
- Staff-protected limited youth profile updates
- Staff role checks through Firebase token verification

Attendance

- Attendance record model
- Staff-protected attendance creation
- Staff-protected attendance retrieval by youth user
- Attendance status values

Service Hours

- Service-hour record model
- Staff-protected service-hour creation/review
- Staff-protected service-hour retrieval by youth user
- Google Form placeholder field

Youth Self-Service

- Protected route for youth to retrieve their own private profile
- Youth profile response includes:
    - Basic profile
    - Earned credentials
    - Attendance records
    - Service-hour records

Authentication and Authorization

- Firebase ID token verification foundation
- Staff/admin role checks
- Staff-only route protection for key backend functions

---

32. Current Backend Flow

The current backend now supports the following logic:

```
None
Youth signs in through Firebase
↓
Youth profile exists in Firestore
↓
Staff can review youth profile
↓
Staff can create credential definition
↓
Staff can award credential
↓
Staff can record attendance
↓
Staff can record service hours
↓
Youth can retrieve their own private profile
↓
Youth profile response includes credentials, attendance, and
service-hour records
```

This is the main September MVP backend flow.

---

33. Current September MVP Readiness

The backend is now ready for staging validation.

It is not yet production-ready.

Ready for Staging

- Backend architecture
- Profile foundation
- Credential backend
- Staff credential awarding
- Staff user management
- Attendance foundation
- Service-hour foundation
- Youth self-service profile retrieval
- Firebase ID token foundation
- Staff role protection
- Local tests

Not Yet Ready for Production

- Frontend integration
- Firebase staging deployment
- Firestore rules review
- Real Firebase Auth testing
- Real Firestore test data
- Production data migration strategy
- Public/legacy endpoint protection review
- Staff dashboard UI
- Youth-facing profile UI
- Credential visual display UI

PART VII: FRONTEND AND STAGING ROADMAP

---

34. Next Development Phase

The next major phase should not be more backend feature expansion.

The next phase should be:

```
None
ASPN September MVP Frontend Integration and Firebase Staging Validation
```

The backend foundation exists. The project now needs to prove that staff and youth can use the platform through the frontend.

---

35. Frontend Integration Priorities

The frontend should be updated to support:

Youth Profile Page

Youth should be able to view:

- Name/profile basics
- Program participation
- Earned credentials
- Attendance records
- Service-hour records

The frontend should call:

```
None
GET /api/me/profile
```

with:

```
None
Authorization: Bearer <Firebase ID token>
```

Staff User Management Page

Staff should be able to:

- View youth users
- Open a youth profile
- Review onboarding status
- Update limited staff-managed fields

Frontend should call:

```
None
GET /api/staff/users/youth
GET /api/staff/users/youth/{id}
PATCH /api/staff/users/youth/{id}
```

Credential Management Page

Staff should be able to:

- Create credential definitions
- Award credentials to youth users

Frontend should call:

```
None
POST /api/staff/credentials/definitions
POST /api/staff/credentials/award
```

Attendance Page

Staff should be able to:

- Create attendance records
- View attendance records for a youth user

Frontend should call:

```
None
POST /api/staff/attendance
GET /api/staff/attendance/user/{userUID}
```

Service-Hour Page

Staff should be able to:

- Create/review service-hour records
- View service-hour records for a youth user

Frontend should call:

```
None
POST /api/staff/service-hours
GET /api/staff/service-hours/user/{userUID}
```

---

36. Required Firebase Staging Setup

Before production launch, staging should include:

Firebase Auth Users

At least:

- One youth user
- One staff user
- One admin user if needed

Firestore User Documents

Each Firebase Auth user must have a matching Firestore document in:

```
None
aspirationnetworkusers
```

The Firestore document ID must equal the Firebase UID.

Required Youth Fields

Youth users should include:

```
None
role = "member"
youthProfile = true
publicProfile = false
profileStatus = "pending_onboarding"
```

Required Staff Fields

Staff users should include:

```
None
role = "staff"
```

or:

```
None
role = "admin"
```

---

37. Staging Test Plan

The first staging test should follow this sequence:

1. Deploy backend to staging.
2. Confirm backend starts successfully.
3. Confirm Firebase credentials work.
4. Create youth test account.
5. Create staff test account.
6. Confirm youth can call:

```
None

GET /api/me/profile
```

7. Confirm youth cannot call staff routes.
8. Confirm staff can list youth users.
9. Confirm staff can create credential definition.

10. Confirm staff can award credential.
11. Confirm staff can create attendance record.
12. Confirm staff can create service-hour record.
13. Confirm youth profile returns:
- Credential
- Attendance
- Service-hour record
14. Test invalid token.
15. Test missing token.
16. Test member role attempting staff action.
17. Confirm forbidden/unauthorized responses work correctly.
18. Review Firestore data after each step.

---

38. Remaining Backend Work After Staging

After staging validation, backend work may include:

- Attendance update/edit workflow
- Youth attendance self-view refinement
- Youth service-hour self-view refinement
- Youth service-hour submission workflow
- Google Form integration
- Credential catalog management
- Program management model
- Role management workflow
- Admin-only role assignment
- Audit logging
- Firestore emulator integration
- Firestore security rules review
- More detailed API documentation
- Public profile design, if ever approved
- Discussion board safety redesign

PART VIII: SAFETY AND PRIVACY NOTES

---

39. Safety Decisions Made During This Phase

The revision process made several safety-first decisions:

1. Youth profiles are private by default.
2. Credential actions are staff-controlled.
3. Youth cannot self-award credentials.
4. Staff routes require Firebase token verification.
5. Staff routes require staff/admin role.
6. Public profile discovery was not built.
7. Peer-to-peer features were not built.
8. Direct messaging was not built.
9. Discussion features were not expanded.
10. Attendance and service-hour records are staff-managed.
11. Youth self-service profile retrieval uses the UID from the verified token rather than accepting user-provided UIDs.

These decisions align with the ASPN platform's safety-by-design direction.

---

40. Privacy Decisions Made During This Phase

The revision process also supported privacy by:

- Keeping youth profiles private.
- Avoiding public youth search.
- Avoiding public credential browsing.
- Avoiding peer-to-peer messaging.
- Restricting staff-level actions.
- Using Firebase identity to retrieve the correct youth profile.
- Avoiding hardcoded credential content.
- Avoiding unnecessary data exposure.

---

41. Risks Still Requiring Attention

Several risks remain before production launch:

Legacy Routes

Some older routes remain unprotected, especially around posts/comments and older profile lookup.

Firestore Rules

Firestore security rules have not yet been fully reviewed in this documentation.

Frontend Token Integration

The frontend must reliably send Firebase ID tokens.

Old User Records

Older Firestore documents may lack new fields.

Staff Role Assignment

Staff/admin roles must be manually or securely assigned.

Testing

Staging testing has not yet been completed.

Production Deployment

The platform should not be deployed directly to production until staging validation succeeds.

PART IX: GITHUB, LOCAL TESTING, AND CODEX WORKFLOW

---

42. Local Testing Workflow

After any future code change, run:

```shell
cd "/Users/colbycrowder/Documents/AspirationsBackendColby/UserData"
./gradlew clean test
```

The expected result is:

```
None
BUILD SUCCESSFUL
```

If the result is:

```
None
BUILD FAILED
```

then the error should be documented and sent back to Codex before proceeding.

---

43. Codex Checkpoint Workflow

Future Codex work should continue using the checkpoint process.

Each checkpoint should require Codex to report:

1. Files changed
2. What changed in plain English
3. Why it matters
4. What youth users can now do
5. What ASPN staff can now do
6. What still has not been built
7. Exact local test command
8. Firebase/staging notes
9. Risks or cautions
10. Frontend requirements if applicable

Codex should not move to the next checkpoint until local verification is complete.

---

44. Recommended GitHub Workflow

Before production launch, the revised code should be committed using a clean Git workflow.

Recommended steps:

```shell
git status
```

Review changed files.

Then:

```shell
git add .
git commit -m "Build September MVP backend foundation"
git push
```

If using branches:

```shell
git checkout -b september-mvp-backend-foundation
git add .
git commit -m "Build September MVP backend foundation"
git push origin september-mvp-backend-foundation
```

Then open a pull request in GitHub.

PART X: MANUAL SUMMARY

---

45. Summary of Completed Phase

The initial ASPN platform prototype began as a Spring Boot/Firebase backend supporting:

- User profiles
- Discussion posts
- Comments
- Upvotes

Through the Codex checkpoint process, it was revised into a September MVP backend foundation supporting:

- Private youth profiles
- Credential definitions
- Earned credential records
- Staff credential awarding
- Credential profile retrieval
- Firebase token verification
- Staff/admin role protection
- Staff user management
- Attendance records
- Service-hour records
- Youth self-service private profile retrieval
- Staging readiness review

This represents a major shift from a basic community/discussion backend toward a youth development and credentialing platform backend.

---

46. Current Platform Identity

The platform should now be understood as:

```
None
ASPN September MVP Backend Foundation
```

It is not yet a finished application.

It is not yet production-ready.

It is ready for:

- Firebase staging deployment
- Frontend integration planning
- Staff workflow testing
- Youth profile testing
- Credential display testing
- Attendance/service-hour staging validation

---

47. Recommended Next Step

The next step should be:

```
None
Frontend Integration and Firebase Staging Validation
```

not additional speculative backend expansion.

The priority should be proving that:

1. A youth can sign in.
2. A youth can retrieve their own profile.
3. Staff can sign in.
4. Staff can review youth users.
5. Staff can create and award credentials.
6. Staff can record attendance.
7. Staff can record service hours.
8. The youth profile displays those records.
9. Youth cannot access staff routes.
10. Staff routes reject missing or invalid tokens.

---

48. Long-Term Future Directions

After the September MVP foundation is validated, future development may include:

- Frontend staff dashboard
- Credential catalog interface
- Attendance editing
- Service-hour submission
- Google Form integration
- Program management
- Cohort management
- Youth project management
- Volunteer tracking
- Scholarship pathways
- Government workforce pathways
- Public-facing project pages
- Limited public credential sharing
- Safe discussion boards
- Moderation workflows
- Audit logs
- Firestore emulator integration
- White-label architecture
- Multi-government scaling

These should be added carefully and only after the core safety architecture is stable.

END OF MANUAL